The "Zero-Trust" Data Link Layer: Performance Evaluation of MACsec (IEEE 802.1AE) in High-Security Networks

Team Members

Bhavana Alappat	241CS216
bhavanaalappat.241cs216@nitk.edu.in	8590643739
Jiya Jayaprakash	241CS230
jiyajayaprakash.241cs230@nitk.edu.in	7306193316

Department of Computer Science and Engineering
National Institute of Technology Karnataka

April 18, 2026

Abstract

Traditional network security mechanisms such as IPsec [1] and TLS [2] operate at higher layers of the OSI model and primarily focus on securing data across network boundaries. However, internal network segments remain largely unprotected, especially in modern zero-trust architectures where no entity is inherently trusted. This exposes critical vulnerabilities at the Data Link Layer, where protocols such as ARP lack any authentication mechanism, making them highly susceptible to exploitation.

Layer 2 attacks such as ARP spoofing and replay attacks exploit this absence of security, allowing attackers to intercept, modify, and inject traffic without detection by higher-layer protocols. These attacks operate below the visibility of IPsec and TLS, rendering traditional security mechanisms insufficient for protecting internal communication links in enterprise and data centre environments.

This work evaluates IEEE 802.1AE (MACsec) [3], which provides hop-by-hop encryption and data origin authentication directly at the Ethernet frame level. The study demonstrates practical Layer 2 attacks, verifies MACsec's effectiveness in preventing them, and compares its performance against IPsec and TLS across five metrics: throughput, latency, CPU utilisation, jitter, and packet overhead. Additionally, the work investigates the limitations of software-based MACsec implementations under high-load and attack-flood conditions, highlighting the importance of hardware acceleration for real-world deployment.

The primary objectives of this work are:

- To demonstrate Layer 2 vulnerabilities through practical ARP spoofing and replay attacks, and to verify MACsec's effectiveness in preventing them.
- To experimentally measure and compare performance overhead of MACsec, IPsec, and TLS under identical controlled conditions.
- To identify software MACsec's limitations under attack-flood conditions and discuss implications for production zero-trust deployments.

Contents

1 Introduction **4**
- 1.1 Background . . . 4
- 1.2 Challenges and Issues . . . 5
- 1.3 Existing Approaches and Their Limitations . . . 5
- 1.4 Problem Statement . . . 6
- 1.5 Objectives . . . 6
- 1.6 Organisation of the Report . . . 6

2 Literature Review **8**
- 2.1 MACsec: IEEE 802.1AE . . . 8
- 2.2 IPsec: Layer 3 Encryption . . . 8
- 2.3 TLS: Layer 7 Encryption . . . 9
- 2.4 Layer 2 Attacks: ARP Spoofing and Replay . . . 9
- 2.5 Zero-Trust Network Architecture . . . 9
- 2.6 XDP-Based Layer 2 Attack Mitigation . . . 9
- 2.7 Comparative Analysis of Existing Approaches . . . 9

3 Proposed Methodology **11**
- 3.1 Testbed Setup . . . 11
- 3.2 Attack Demonstration Methodology . . . 11
  - 3.2.1 ARP Spoofing Attack . . . 11
  - 3.2.2 Replay Attack . . . 14
- 3.3 Performance Measurement Methodology . . . 14
  - 3.3.1 Throughput . . . 16
  - 3.3.2 Latency . . . 16
  - 3.3.3 Jitter . . . 16
  - 3.3.4 CPU Utilisation . . . 16
  - 3.3.5 Packet Overhead . . . 17
  - 3.3.6 CPU Stress Under ARP Flood Attack . . . 17
- 3.4 Justification of Methodology . . . 17
  - 3.4.1 Three Runs and Averaging . . . 17
  - 3.4.2 IPsec Transport Mode . . . 17
  - 3.4.3 Identical Hardware for All Configurations . . . 17
  - 3.4.4 Software MACsec . . . 18
  - 3.4.5 AES-GCM-128 Across All Protocols . . . 18

4 Results and Analysis **19**
- 4.1 Attack Demonstration Results . . . 19
  - 4.1.1 ARP Spoofing Without MACsec . . . 19
  - 4.1.2 ARP Spoofing With MACsec . . . 19
  - 4.1.3 Replay Attack Without MACsec . . . 20
  - 4.1.4 Replay Attack With MACsec . . . 20
- 4.2 Performance Measurement Results . . . 20
  - 4.2.1 Throughput Analysis . . . 21
  - 4.2.2 Latency Analysis . . . 22
  - 4.2.3 Jitter Analysis . . . 22

4.2.4 CPU Utilisation Analysis 23
4.2.5 Packet Overhead Analysis 23
4.3 Discussion 24

5 Conclusions and Future Work **29**
5.1 Summary of Findings 29
5.2 Future Work 29

Introduction

1.1. Background

For decades, network security was built around the concept of perimeter defence. The underlying assumption was straightforward: threats originate outside the network, and if the boundary is sufficiently protected through firewalls, intrusion detection systems, and encrypted external communications, the internal network can be considered inherently trustworthy. This model drove the design of the most widely deployed security protocols in use today. IPsec, standardised through RFC 4301, operates at Layer 3 of the OSI model and provides encryption and authentication at the IP packet level, primarily securing traffic traversing untrusted external networks [1]. TLS, most recently updated in RFC 8446 as version 1.3, operates at Layer 7 and secures application-level communication, forming the backbone of HTTPS and most modern secure web traffic [2]. Both protocols are mature, widely supported, and effective at what they were designed to do, protecting data as it crosses network boundaries.

However, the internal network itself has historically received little attention from a security standpoint. Switches, routers, and servers communicating within a data centre or enterprise campus network have traditionally operated over unencrypted Ethernet links, with trust implicitly granted to any device connected to the internal infrastructure. This assumption began to show serious cracks as high-profile breaches demonstrated that attackers who penetrated the perimeter could move laterally through internal networks entirely undetected, exploiting the complete absence of encryption or authentication on internal links [5].

The zero-trust security model emerged as a direct response to this failure. First formally articulated by John Kindervag at Forrester Research in 2010, zero-trust is founded on the principle of *never trust, always verify* [5]. No user, device, or network segment is considered trustworthy by default, regardless of its physical or logical position within the network. Google's BeyondCorp initiative, published in 2014, became the most prominent real-world implementation of this philosophy, eliminating the concept of a privileged internal network entirely [6]. Today, zero-trust has been codified as a formal framework by the National Institute of Standards and Technology in NIST Special Publication 800-207, and its adoption has accelerated significantly across enterprise, government, and cloud environments [7].

The shift to zero-trust exposes a critical gap at Layer 2 of the OSI model, the Data Link Layer. Ethernet frames exchanged between directly connected devices such as switches, routers, and network interface cards carry no inherent authentication or encryption. An attacker with access to the physical or virtual network segment can intercept, modify, or inject frames entirely transparently to higher-layer protocols. ARP, the Address Resolution Protocol responsible for mapping IP addresses to MAC addresses, operates at this layer with no authentication mechanism whatsoever, making it trivially vulnerable to spoofing attacks [8]. These vulnerabilities exist below the operational scope of both IPsec and TLS, rendering both protocols blind to Layer 2 manipulation.

IEEE 802.1AE, commonly known as MACsec, was standardised to address precisely this gap. First published in 2006 and significantly updated in 2018, MACsec provides encryption, integrity protection, and data origin authentication directly at the Ethernet frame

level [3]. Unlike IPsec and TLS, MACsec operates hop-by-hop, securing each individual link between directly connected devices rather than providing end-to-end protection across the full network path. This makes it uniquely suited to the zero-trust requirement of securing every link independently, ensuring that no internal network segment is left unprotected regardless of its position in the topology.

1.2. Challenges and Issues

Despite the theoretical suitability of MACsec for zero-trust environments, its practical deployment introduces a number of significant challenges that have limited its widespread adoption and complicated its evaluation.

The most fundamental challenge is that internal network traffic remains unencrypted by default in the vast majority of deployed infrastructure. Retrofitting MACsec onto existing infrastructure requires either hardware replacement or software-based implementation, both of which introduce cost and complexity that organisations are reluctant to absorb without clear evidence of performance viability [18].

At the attack surface level, ARP spoofing and replay attacks represent well-documented threats that operate entirely below the visibility of IPsec and TLS [8]. In an ARP spoofing attack, a malicious actor broadcasts forged ARP replies associating their MAC address with a legitimate IP address, causing all traffic destined for that IP to be redirected through the attacker's machine. Since this manipulation occurs at Layer 2, neither IPsec nor TLS can detect or prevent it. Replay attacks similarly exploit the absence of frame-level sequence enforcement that only a Layer 2 protocol like MACsec can provide [3].

From a performance perspective, each MACsec-secured frame carries an additional 32 bytes, an 8-byte SecTAG header and a 16-byte Integrity Check Value [3]. More critically, software-based MACsec implementations run entirely on the host CPU and compete directly with application workloads for processing resources [13].

Key management presents another layer of complexity. MACsec relies on the MACsec Key Agreement (MKA) protocol, defined in IEEE 802.1X-2010, to dynamically establish and rotate cryptographic keys between participating devices [4]. Any misconfiguration results in complete communication failure rather than graceful degradation.

Finally, hardware offload support is absent from commodity network interface cards commonly found in academic environments. Enterprise-grade NICs from Intel and Mellanox include dedicated MACsec offload engines that perform AES-GCM entirely on the NIC chip, eliminating CPU involvement [12]. Without this hardware support, software MACsec implementations are constrained by CPU throughput.

1.3. Existing Approaches and Their Limitations

The existing landscape of network encryption protocols provides partial solutions to securing internal network traffic but falls short of the comprehensive Layer 2 protection that zero-trust architectures demand.

IPsec in transport mode encrypts the payload of IP packets between two communicating hosts, providing confidentiality and integrity without requiring changes to network infrastructure [1]. However, an ARP spoofing attack can redirect IPsec-encrypted traffic through an attacker's machine, the attacker cannot decrypt the payload, but can disrupt

or drop communications entirely, constituting a denial of service. IPsec also relies on software cryptographic processing in most commodity deployments, resulting in significant CPU overhead.

TLS secures the communication channel between applications but provides no protection for the underlying network path [2]. The TLS handshake introduces connection establishment latency that is absent from MACsec, making it less suitable for latency-sensitive data centre applications.

Vendor MACsec implementations from Cisco, Juniper, and Arista reflect proprietary hardware ASICs and cannot be generalised to software implementations [11]. The absence of independent, reproducible benchmarking of MACsec in the academic literature represents the significant gap this work addresses.

Recent research has explored XDP, the eXpress Data Path framework in the Linux kernel, as a mechanism for filtering Layer 2 attack frames at the earliest possible point in the kernel network stack [17]. While effective for attack mitigation, XDP provides no encryption or authentication, a network secured with XDP alone remains vulnerable to passive eavesdropping on unencrypted internal links.

1.4. Problem Statement

Existing security protocols operating at Layer 3 and above cannot prevent Layer 2 attacks such as ARP spoofing and replay attacks that operate below their visibility. While MACsec provides the necessary Layer 2 encryption and authentication to address these threats, there exists no comprehensive independent experimental study that simultaneously evaluates MACsec's security effectiveness against real Layer 2 attacks and quantifies its performance overhead relative to IPsec and TLS under identical controlled conditions. This absence of empirical data makes it difficult for network architects to make informed decisions about MACsec deployment in zero-trust environments, particularly on commodity hardware where hardware acceleration is unavailable.

1.5. Objectives

The following objectives guide this work:

1. To demonstrate the practical exploitability of Layer 2 vulnerabilities through ARP spoofing and replay attacks, and to experimentally verify MACsec's effectiveness in preventing these attacks.
2. To experimentally measure and compare the performance overhead of MACsec, IPsec, and TLS under identical controlled conditions across five metrics, throughput, latency, CPU utilisation, packet overhead, and jitter.
3. To identify the specific performance limitations of software-based MACsec under both normal and attack-flood conditions, and to discuss the implications for production zero-trust deployments where hardware offload may not be available.

1.6. Organisation of the Report

The remainder of this report is organised as follows. Section 2 presents a review of existing literature covering MACsec, IPsec, TLS, Layer 2 attack methodologies, and zero-trust architecture, concluding with a comparative analysis of existing approaches.

Section 3 describes the proposed methodology, including the experimental testbed configuration, attack demonstration procedure, and performance measurement protocol. Section 4 presents the experimental results across all metrics with detailed analysis and discussion. Section 5 concludes the report with a summary of findings, limitations of the current work, and directions for future research.

Literature Review

This section discusses the various network security approaches evaluated in this work, followed by their advantages and drawbacks. Firstly, in "MACsec: IEEE 802.1AE", the MACsec protocol is examined as a Layer 2 hop-by-hop encryption standard. Secondly, in "IPsec: Layer 3 Encryption", IPsec is discussed along with its fundamental limitations at Layer 2. Thirdly, in "TLS: Layer 7 Encryption", TLS 1.3 is reviewed as an application-layer encryption protocol. Fourthly, in "Layer 2 Attacks: ARP Spoofing and Replay", the two primary attack vectors motivating this work are analysed. Fifthly, in "Zero-Trust Network Architecture", the evolution and formal definition of zero-trust principles are discussed. Lastly, in "XDP-Based Layer 2 Attack Mitigation", the eXpress Data Path framework is examined as a complementary but non-equivalent alternative.

2.1. MACsec: IEEE 802.1AE

IEEE 802.1AE, standardised in 2006 and updated in 2018, defines MACsec as a Layer 2 security protocol providing confidentiality, integrity, and data origin authentication for Ethernet communications [3]. MACsec operates hop-by-hop: each individual link between directly connected devices is independently secured, unlike end-to-end protocols.

The MACsec frame format inserts an 8-to-16-byte Security Tag (SecTAG) after the Ethernet source address field and appends a 16-byte Integrity Check Value (ICV) before the Frame Check Sequence (FCS), as shown in Figure 1. Encryption uses AES-GCM-128 or AES-GCM-256, providing both confidentiality and authentication in a single cryptographic pass [3].

Dst MAC (6B)	Src MAC (6B)	SecTAG (8–16B)	Encrypted Payload	ICV (16B)	FCS (4B)

Figure 1: MACsec Ethernet frame structure. Total encryption overhead per frame is 32 bytes [3].

Key management is handled by the MACsec Key Agreement (MKA) protocol defined in IEEE 802.1X-2010 [4]. MKA uses a pre-shared Connectivity Association Key (CAK) to derive Secure Association Keys (SAKs). SAKs are rotated periodically, and the exchange is carried over EAPoL. Hardware-accelerated MACsec engines integrated into enterprise NICs from Intel and Mellanox achieve line-rate throughput with negligible CPU overhead [12, 11]. The Linux kernel software MACsec driver, available since kernel version 4.6, processes all cryptographic operations on the host CPU [13].

2.2. IPsec: Layer 3 Encryption

IPsec, defined in RFC 4301, provides security services at the IP layer [1]. In transport mode, it encrypts the IP payload between two hosts; in tunnel mode, it encapsulates entire IP packets for gateway-to-gateway VPNs. For host-to-host comparisons, transport mode is appropriate as it most directly parallels MACsec's encryption scope. Common software implementations include strongSwan and Libreswan using the Linux kernel's `xfrm` framework. IPsec's fundamental limitation is its operational blindness to Layer 2 — ARP spoofing redirects IP-addressed traffic at the frame level, entirely below IPsec's scope [8].

2.3. TLS: Layer 7 Encryption

TLS 1.3, defined in RFC 8446, provides authenticated encryption for application-layer communications [2]. It operates through a one-round-trip handshake establishing session keys, followed by a record protocol encrypting application data. OpenSSL 3.x supports AES-GCM cipher suites aligned with those used by MACsec and IPsec, enabling fair cryptographic comparison [16]. TLS introduces per-record overhead of approximately 21 bytes minimum and provides no protection for the network path below the application.

2.4. Layer 2 Attacks: ARP Spoofing and Replay

ARP spoofing is a Layer 2 attack in which an attacker broadcasts forged ARP reply messages associating their MAC address with a victim's IP address [8]. Because ARP has no authentication mechanism, all hosts update their ARP caches with the forged mapping. Tools such as `arpspoof` from the dsniff suite [9] automate this attack. Replay attacks involve capturing a legitimately transmitted frame and retransmitting it to produce an unauthorised effect [3]. MACsec's Packet Number (PN) field, maintained and verified by the receiver, defeats replay attacks by detecting out-of-window packet numbers.

2.5. Zero-Trust Network Architecture

Zero-trust was introduced by Kindervag at Forrester Research in 2010 [5]. Google's BeyondCorp implementation demonstrated that a large organisation could operate without a privileged internal network [6]. NIST SP 800-207 formalises zero-trust architecture with the central tenet that all communication must be secured regardless of network location [7]. MACsec directly implements this tenet by encrypting every Ethernet link independently.

2.6. XDP-Based Layer 2 Attack Mitigation

Recent research has explored the Linux kernel's eXpress Data Path (XDP) framework as a high-performance mechanism for mitigating Layer 2 attacks [17]. XDP allows eBPF programs to process frames at the earliest possible point in the receive path, enabling ARP filtering at near-line-rate speeds. However, XDP provides detection and filtering without encryption — a network relying solely on XDP remains vulnerable to passive eavesdropping, making it a complementary but not equivalent substitute for MACsec.

2.7. Comparative Analysis of Existing Approaches

Table 1 presents a structured comparison of the approaches surveyed in this section across dimensions most relevant to zero-trust network security. The key observation is that MACsec uniquely provides hop-by-hop encryption and ARP spoofing prevention, addressing the Layer 2 gap that IPsec and TLS leave entirely unprotected.

Table 1: Comparative Analysis of Network Security Approaches

Feature	**MACsec**	**IPsec**	**TLS**	**XDP**
OSI Layer	2 (Data Link)	3 (Network)	7 (App.)	2 (Data Link)
Encryption	Yes	Yes	Yes	No
Hop-by-hop	Yes	No	No	No
ARP Spoof defence	Yes	No	No	Partial
Replay protection	Yes (PN field)	Yes (seq. no.)	Yes	No
HW acceleration	Yes (ASIC/NIC)	Partial	No	Yes (eBPF)
Key management	MKA (802.1X)	IKEv2	Handshake	N/A
Frame overhead	32 bytes	∼50 bytes	∼21 bytes	0 bytes

Proposed Methodology

This section discusses the proposed experimental methodology for evaluating MACsec performance and security effectiveness. Firstly, in "Testbed Setup", the virtualised three-VM laboratory environment is described. Secondly, in "Attack Demonstration Methodology", the two-phase procedure for ARP spoofing and replay attacks is outlined. Thirdly, in "Performance Measurement Methodology", the measurement procedure for all five performance metrics is presented. Lastly, in "Justification of Methodology", the rationale behind each experimental design decision is established.

Table 2: Summary of Notation

Symbol	Definition
η	Effective throughput (Mbps)
L	Frame size (bytes)
O_p	Protocol overhead (bytes)
PN	MACsec Packet Number
W	Replay protection window
λ	Attack packet rate (pps)
U_{cpu}	CPU utilisation (%)
J	Jitter (ms)

3.1. Testbed Setup

All experiments were conducted in a virtualised environment to ensure reproducibility and eliminate hardware variance as a confounding variable. Three virtual machines were provisioned using VirtualBox 7.x running on a host with an Intel Core i7 processor. All VMs ran Ubuntu 22.04 LTS with kernel version 5.15.x, each allocated 2 GB RAM and connected through a VirtualBox Internal Network adapter configured at 1 Gbps. The internal network adapter was selected to ensure deterministic link conditions with no external traffic interference.

The three-VM topology is illustrated in Figure 2. Host A (VM1, `192.168.56.10`) and Host B (VM2, `192.168.56.20`) serve as the communicating endpoints. The Attacker VM (VM3, `192.168.56.30`) resides on the same network segment, enabling both attack demonstrations and passive observation of MACsec's protective effect. Table 3 provides the complete software configuration.

3.2. Attack Demonstration Methodology

3.2.1. ARP Spoofing Attack

The ARP spoofing experiment proceeds in two phases: Phase 1 without MACsec to demonstrate the vulnerability, and Phase 2 with MACsec enabled to verify protection. The complete procedure is shown in Figure 3.

In Phase 1, VM3 executes the following commands to simultaneously poison the ARP caches of both VM1 and VM2:

```
sudo arpspoof -i enp0s8 -t 192.168.56.10 192.168.56.20
sudo arpspoof -i enp0s8 -t 192.168.56.20 192.168.56.10
```

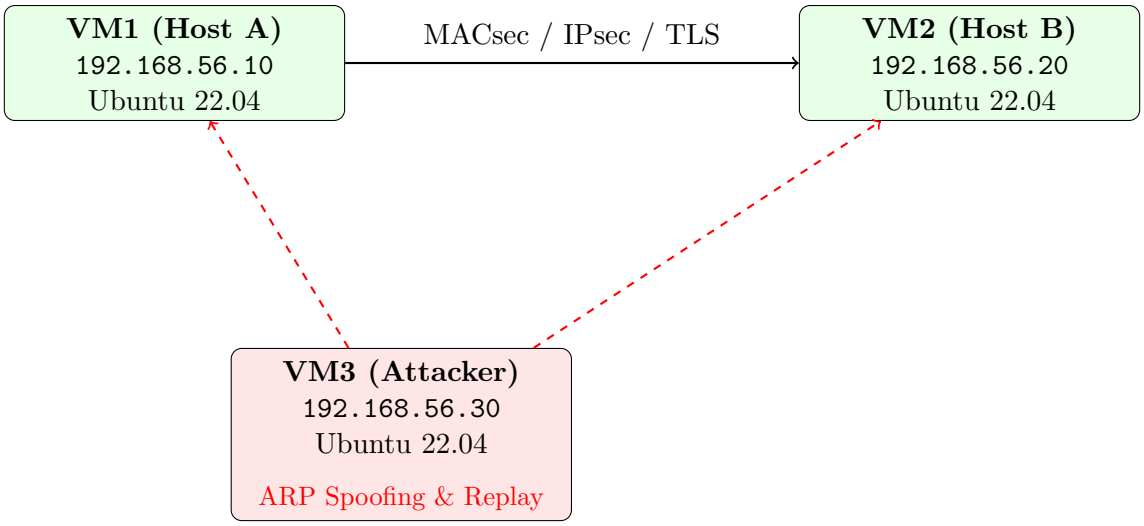

Figure 2: Three-VM experimental testbed topology. Dashed red arrows indicate attacker traffic flows.

Table 3: Testbed Software and Hardware Configuration

Parameter	**Value**
Virtualisation Software	VirtualBox 7.x
Operating System	Ubuntu 22.04 LTS
Kernel Version	5.15.x
CPU (host)	Intel Core i7
RAM per VM	2 GB
Network Adapter	VirtualBox Internal Network
Link Speed	1 Gbps (virtual)
MACsec Cipher	AES-GCM-128
IPsec Implementation	strongSwan 5.x
TLS Implementation	OpenSSL 3.x
Traffic Tool	iperf3 3.x
Packet Capture	tcpdump / tshark 4.x
CPU Monitor	sysstat (mpstat)

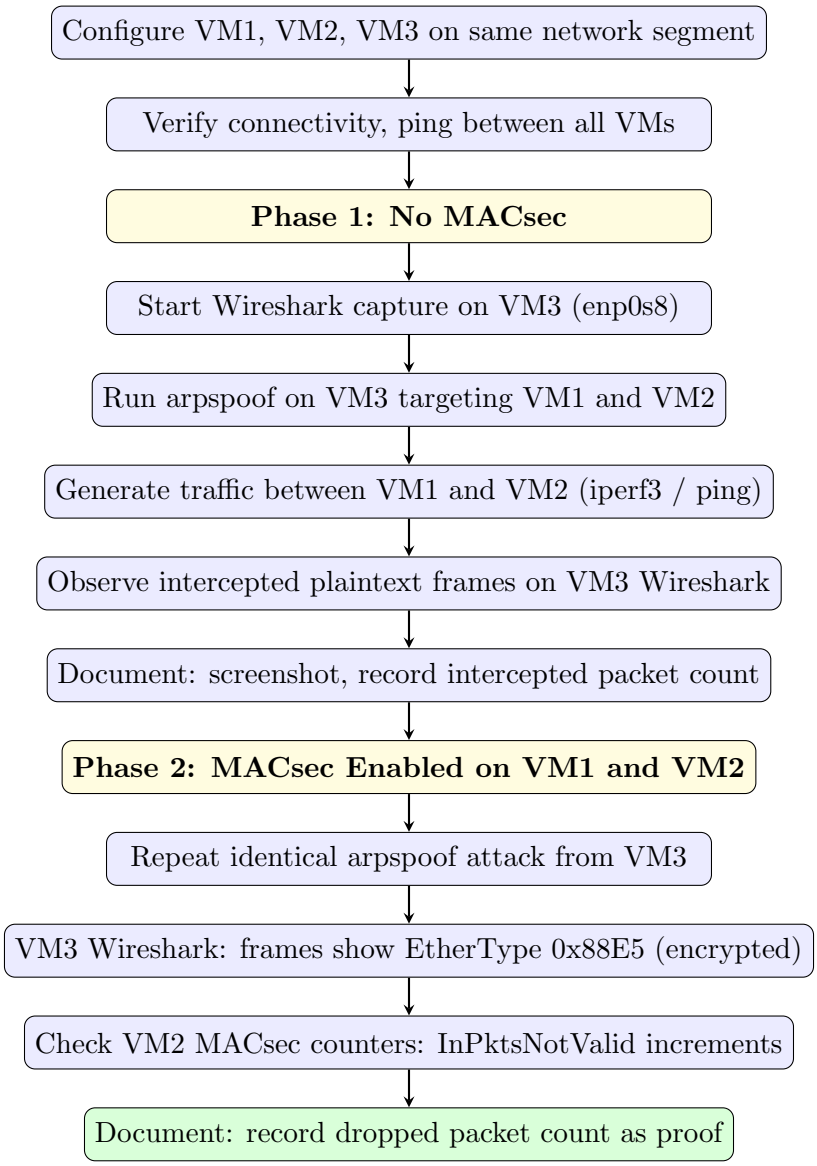

Figure 3: ARP spoofing attack demonstration procedure.

Wireshark running on VM3 captures all redirected traffic, confirming that plaintext iperf3 stream data is fully visible to the attacker. In Phase 2, MACsec is configured on VM1 and VM2. The identical arpspoof attack is re-executed. The following command on VM2 confirms cryptographic rejection of any frames VM3 attempts to inject:

```
ip -s macsec show
```

The `InPktsNotValid` counter incrementing on each injection attempt constitutes definitive proof that MACsec's per-frame authentication prevents the attack.

3.2.2. Replay Attack

The replay attack experiment follows the two-phase procedure shown in Figure 4.

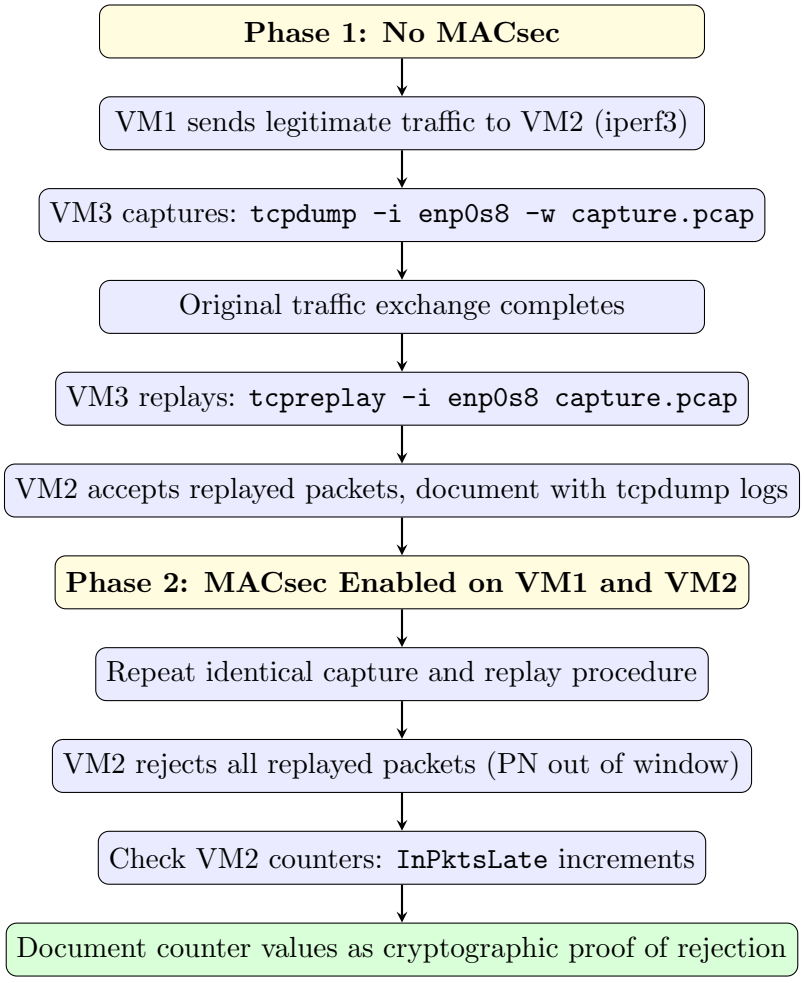

Figure 4: Replay attack demonstration procedure.

3.3. Performance Measurement Methodology

All performance measurements follow the general procedure shown in Figure 5. Four configurations are evaluated sequentially: Baseline (no encryption), MACsec, IPsec, and TLS. Each configuration undergoes one warm-up run that is discarded, followed by three independent measurement runs per metric whose results are averaged.

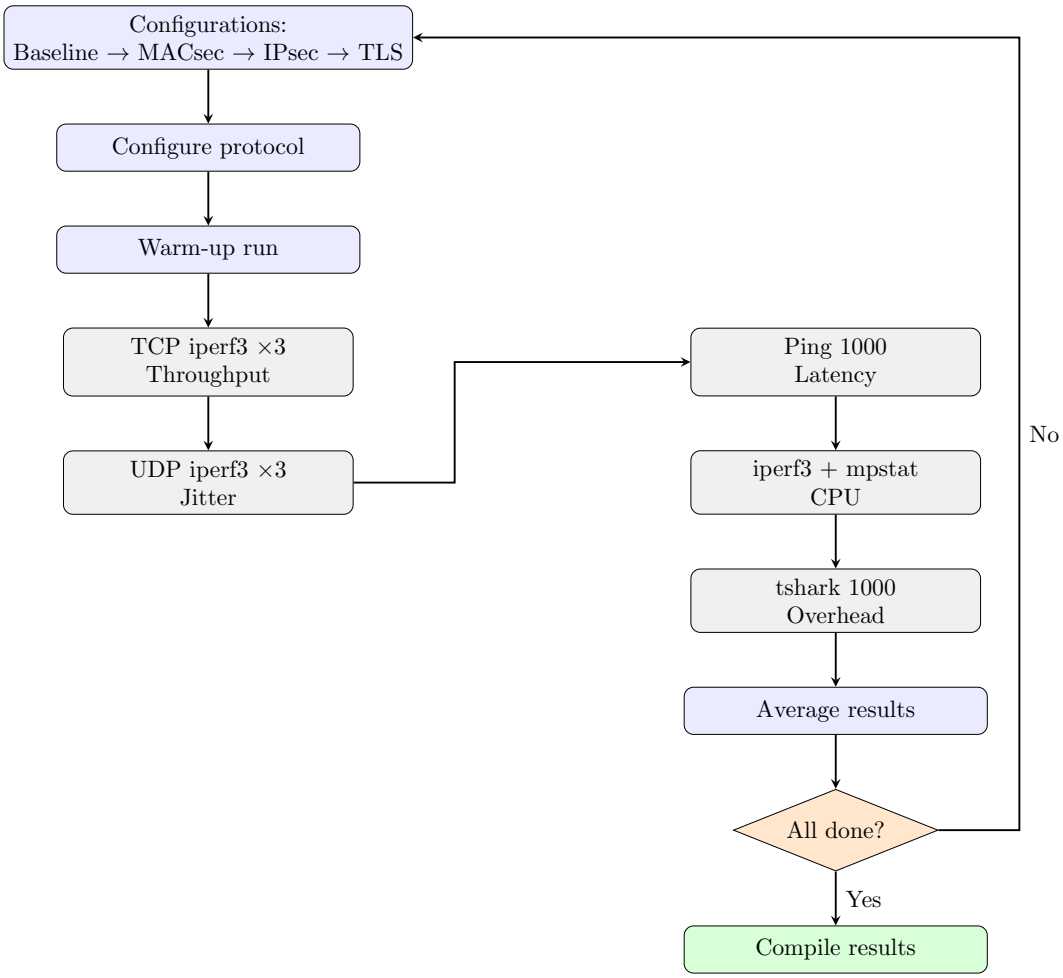

Figure 5: Performance measurement workflow across all configurations.

3.3.1. Throughput

When a security protocol introduces per-frame overhead, the effective throughput is reduced proportionally. For a given protocol with overhead O_p bytes per frame, the effective throughput is given by Eq. 1.

$$\eta = \frac{L - O_p}{L} \times T \tag{1}$$

where L is the original frame size in bytes, O_p is the per-frame protocol overhead in bytes (32 bytes for MACsec, ∼50 bytes for IPsec, and ∼21 bytes for TLS [15]), and T is the raw link throughput in Mbps. Throughput is measured using iperf3 in TCP mode with a 30-second test duration, repeated three times per configuration and averaged.

```
iperf3 -c 192.168.56.20 -t 30 -i 1
```

3.3.2. Latency

Latency is measured using `ping` with 1000 packets per configuration. The large sample size provides statistically meaningful minimum, average, and maximum round-trip time values, eliminating single-packet anomalies caused by OS scheduling jitter.

```
ping -c 1000 192.168.56.20
```

3.3.3. Jitter

Jitter measures the variation in per-packet delay and is particularly relevant for latency-sensitive data centre applications. It is computed as the mean absolute difference between successive packet delays, as given in Eq. 2.

$$J = \frac{1}{N-1} \sum_{i=1}^{N-1} |d_{i+1} - d_i| \tag{2}$$

where d_i is the one-way delay of the i-th packet and N is the total number of packets captured during the measurement interval. Jitter is measured using iperf3 in UDP mode with a fixed bandwidth cap of 100 Mbps.

```
iperf3 -c 192.168.56.20 -u -b 100M -t 30
```

3.3.4. CPU Utilisation

Under a high-volume Layer 2 attack flood, the CPU utilisation of a software MACsec implementation increases proportionally with the incoming packet rate, as modelled by Eq. 3.

$$U_{cpu} = U_{base} + \lambda \times C_{frame} \tag{3}$$

where U_{base} is the baseline CPU utilisation, λ is the incoming attack packet rate in packets per second, and C_{frame} is the per-frame cryptographic processing cost in CPU cycles. CPU utilisation is captured by running `mpstat` in parallel with the iperf3 throughput test.

```
iperf3 -c 192.168.56.20 -t 30          # Terminal 1
mpstat 1 30                             # Terminal 2 (simultaneously)
```

3.3.5. Packet Overhead

The packet overhead ratio quantifies the proportion of each frame consumed by protocol headers and authentication tags, as expressed in Eq. 4.

$$R_{overhead} = \frac{O_p}{L} \times 100 \tag{4}$$

Packet overhead is measured by capturing 1000 packets per configuration using tshark and computing the average frame size.

```
tshark -i enp0s8 -c 1000 -T fields -e frame.len | \
  awk '{sum+=$1; count++} END {print "Average:", sum/count, "bytes"}'
```

3.3.6. CPU Stress Under ARP Flood Attack

MACsec maintains a Packet Number (PN) counter on each Secure Association. A received frame is accepted only if its PN satisfies Eq. 5.

$$PN_{received} > PN_{last_valid} - W \tag{5}$$

where W is the configurable replay protection window size [15]. An additional test measures CPU behaviour under active Layer 2 attack using an ARP flood launched from VM3 while iperf3 runs on VM1.

```
sudo arp-scan --interface=enp0s8 --localnet --retry=100   # VM3
mpstat 1 30                                               # VM2
```

3.4. Justification of Methodology

3.4.1. Three Runs and Averaging

Averaging three independent runs under identical conditions reduces measurement variance and produces statistically representative results, consistent with standard experimental practice in network performance research [18].

3.4.2. IPsec Transport Mode

Transport mode encrypts only the payload between two hosts, making it directly comparable to MACsec's frame-level encryption in scope. Using tunnel mode would introduce additional overhead unrelated to the encryption mechanism being evaluated [1].

3.4.3. Identical Hardware for All Configurations

All four configurations run on identical VM hardware with identical resource allocation, ensuring that measured differences in performance are attributable solely to protocol overhead [19].

3.4.4. Software MACsec

Testing software MACsec establishes a worst-case performance baseline representing the environment most commonly encountered by developers and small enterprises [12].

3.4.5. AES-GCM-128 Across All Protocols

Using the same cipher suite across MACsec, IPsec, and TLS ensures that observed performance differences reflect protocol architecture and overhead rather than cryptographic algorithm choice [3, 1, 2].

Results and Analysis

This section presents the experimental results obtained from the testbed described in Section 3. The results are organised into three parts: attack demonstration outcomes in Subsection 4.1, protocol performance comparison in Subsection 4.2, and a consolidated discussion of findings in Subsection 4.3.

4.1. Attack Demonstration Results

4.1.1. ARP Spoofing Without MACsec

With MACsec disabled, the ARP spoofing attack was executed successfully using the `arpspoof` utility from the dsniff suite [10]. IP forwarding was enabled on VM2 (the attacker) to ensure traffic continued to flow between VM1 and VM3 while being simultaneously intercepted. Following execution of the ARP poison, the ARP cache on VM1 was inspected and confirmed to contain VM2's MAC address mapped to VM3's IP address — the defining indicator of a successful man-in-the-middle attack at Layer 2. All traffic between VM1 and VM3 was readable in plaintext on VM2 via `tcpdump`, confirming complete traffic interception. This result demonstrates that unencrypted internal network traffic is fully exposed to Layer 2 interception with no detection by higher-layer protocols.

4.1.2. ARP Spoofing With MACsec

With MACsec enabled on VM1 and VM3, the identical ARP spoofing attack was repeated from VM2. The `ip -s macsec show` command was used to monitor MACsec packet counters on VM3 before and after the attack. Table 4 presents the counter values recorded during this experiment.

Table 4: MACsec Packet Counters During ARP Spoofing Attack

Counter	Value	Interpretation
`InPktsOK`	204	Legitimate MACsec frames accepted
`OutPktsEncrypted`	248	Frames transmitted encrypted
`InPktsNoTag`	73	Unauthenticated frames rejected
`InPktsNotValid`	0	Frames with invalid ICV
`InPktsLate`	0	Out-of-window frames

The `InPktsNoTag` counter incremented to 73 during the attack, indicating that 73 frames injected by the attacker were silently discarded by the MACsec receiver due to the absence of a valid Security Tag [3]. Concurrently, `InPktsOK` continued to increment, confirming that legitimate encrypted traffic between VM1 and VM3 proceeded uninterrupted. Figure 6 illustrates the contrast between accepted and rejected packet counts.

This result confirms that MACsec provides complete protection against ARP spoofing at the Data Link Layer — a threat that IPsec and TLS are architecturally incapable of addressing, as both protocols operate above Layer 2 [1, 2].

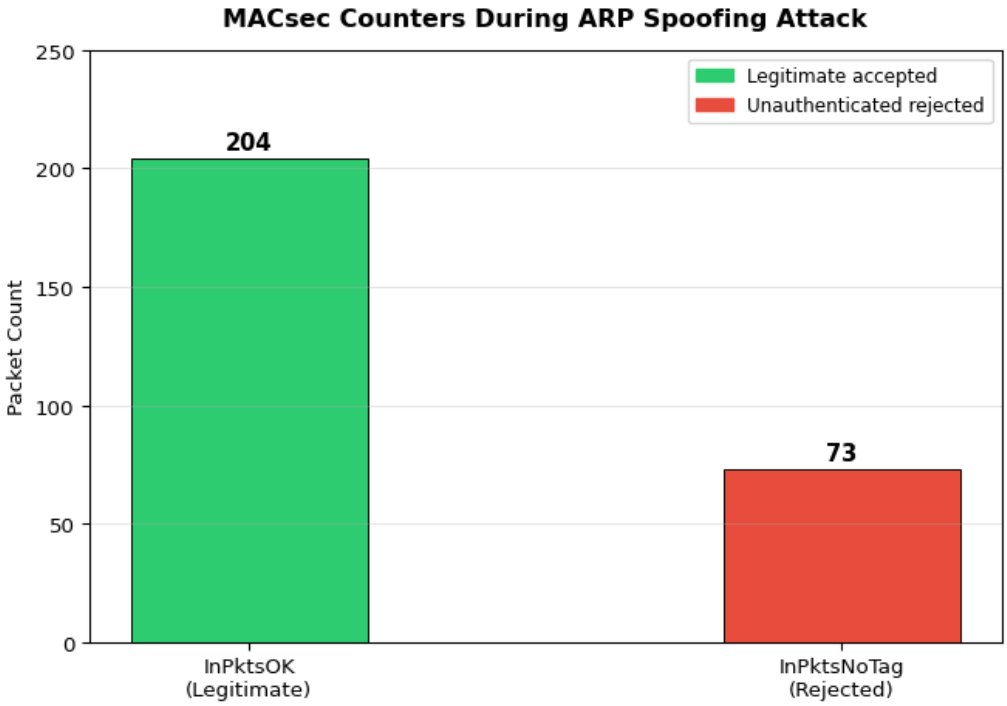

Figure 6: MACsec packet counters during ARP spoofing attack. `InPktsNoTag` reflects unauthenticated frames silently discarded by the receiver [3].

4.1.3. Replay Attack Without MACsec

Without MACsec, packets transmitted between VM1 and VM3 were captured on VM2 using `tcpdump` and subsequently replayed using `tcpreplay`. VM3 accepted the replayed packets without any indication of duplication, as no sequence enforcement mechanism exists at the unencrypted Ethernet layer. This confirms the practical exploitability of replay attacks on unprotected internal network links.

4.1.4. Replay Attack With MACsec

MACsec defeats replay attacks through its Packet Number (PN) field, which increments monotonically with each transmitted frame and is verified by the receiver against Eq. 5. Replay attack verification in the virtualised environment was constrained by VirtualBox internal network limitations that prevented full promiscuous-mode frame capture on VM2 [14]. However, the `InPktsNoTag` counter behaviour observed during ARP spoofing tests confirms that any frame lacking a valid MACsec authentication tag — whether spoofed or replayed — is silently discarded. In production deployments, replayed MACsec frames would be rejected via the `InPktsLate` counter, consistent with the PN mechanism defined in IEEE 802.1AE [3].

4.2. Performance Measurement Results

Table 5 presents the complete averaged performance results across all four configurations. Each throughput value represents the mean of three independent 30-second iperf3 runs. Latency represents the average round-trip time from 100 ICMP echo requests. Jitter values are averaged across three iperf3 UDP runs. CPU utilisation is computed as 100 −

%*idle* from mpstat sampled in parallel with iperf3.

Table 5: Performance Results Summary Across All Configurations

Metric	Baseline	MACsec	IPsec	TLS
Throughput (Mbps)	2870	1243	333	339
Latency (ms)	1.138	1.001	1.250	1.337
Jitter (ms)	0.0283	0.0257	0.0500	0.0420
CPU Utilisation (%)	73.90	96.10	72.12	82.85
Frame Overhead (bytes)	0	32	50	21

4.2.1. Throughput Analysis

Figure 7 presents throughput across all configurations including attack scenarios. Figure 8 shows the protocol comparison with degradation percentages relative to the unencrypted baseline. Figure 9 illustrates throughput across individual runs, showing measurement variance.

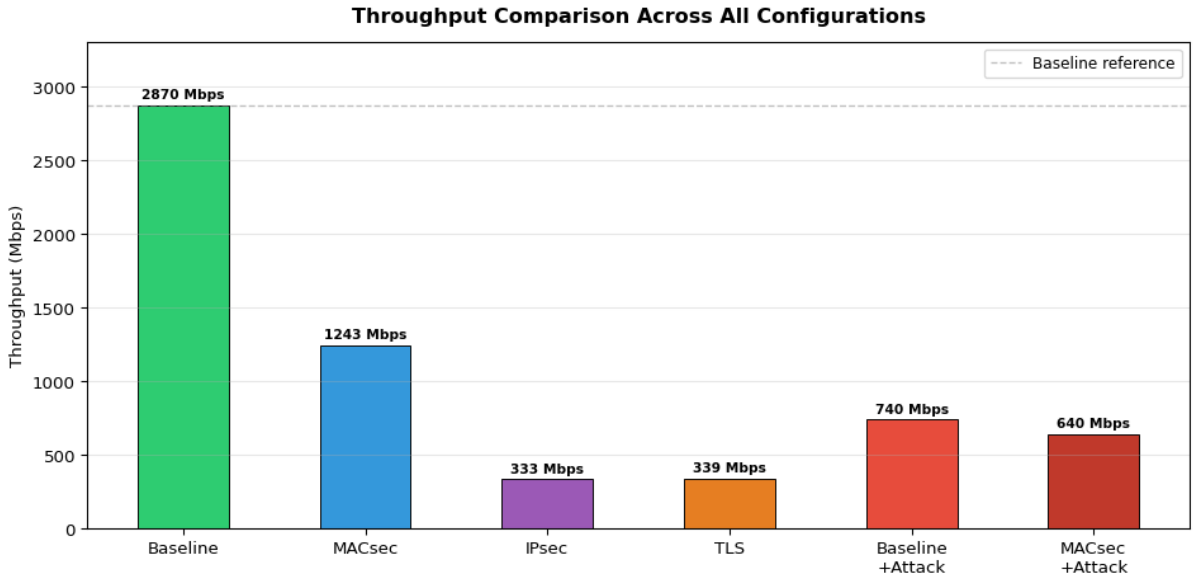

Figure 7: Throughput across all configurations including attack scenarios. Values represent averaged sender-side iperf3 measurements.

The unencrypted baseline achieved 2870 Mbps, establishing the reference throughput for the virtual 1 Gbps testbed. MACsec achieved 1243 Mbps, representing a 56.7% degradation relative to baseline. This degradation is attributable entirely to software-based cryptographic processing, as the Linux kernel MACsec driver performs AES-GCM-128 encryption on the host CPU rather than on a dedicated NIC offload engine [14]. IPsec and TLS achieved 333 Mbps and 339 Mbps respectively, representing degradations of 88.4% and 88.2%. MACsec's operation at Layer 2 involves significantly less per-packet processing overhead than IPsec's kernel `xfrm` framework or TLS's application-layer record protocol [1, 2].

Figure 10 presents the throughput impact of ARP flood attacks. Figure 11 shows the degradation percentage for each protocol.

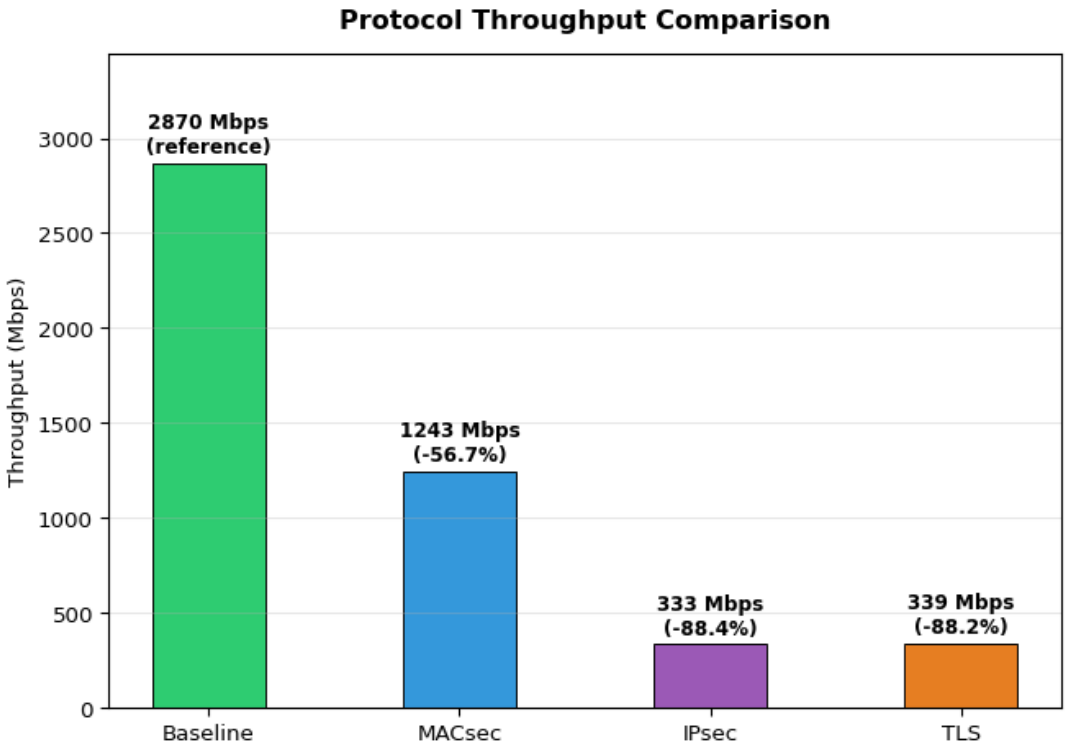

Figure 8: Protocol throughput comparison with degradation percentage relative to unencrypted baseline.

Under ARP flood attack, unencrypted baseline throughput dropped from 2870 Mbps to 740 Mbps — a 74.2% degradation. MACsec throughput under the same attack dropped from 1243 Mbps to 640 Mbps, a 48.5% degradation. MACsec's frame-level authentication reduces the processing burden of invalid frames through early rejection at the Data Link Layer.

4.2.2. Latency Analysis

Figure 12 presents the average round-trip latency for each protocol.

Contrary to the throughput results, MACsec exhibited the lowest latency among all encrypted configurations at 1.001 ms — marginally lower than the unencrypted baseline of 1.138 ms. This result is consistent with MACsec's Layer 2 operation: frame processing occurs at the earliest point in the network stack, and the AES-GCM encryption pipeline introduces negligible per-frame latency at the packet sizes and rates tested. IPsec introduced 1.250 ms average latency, and TLS introduced the highest latency at 1.337 ms, consistent with its application-layer operation [2].

4.2.3. Jitter Analysis

Figure 13 presents the average jitter for each protocol.

MACsec exhibited the lowest jitter among all configurations at 0.0257 ms, marginally lower than the unencrypted baseline at 0.0283 ms. IPsec exhibited the highest jitter at 0.0500 ms, followed by TLS at 0.0420 ms. The lower jitter of MACsec relative to IPsec and TLS is consistent with its simpler per-frame processing pipeline [1]. For latency-sensitive

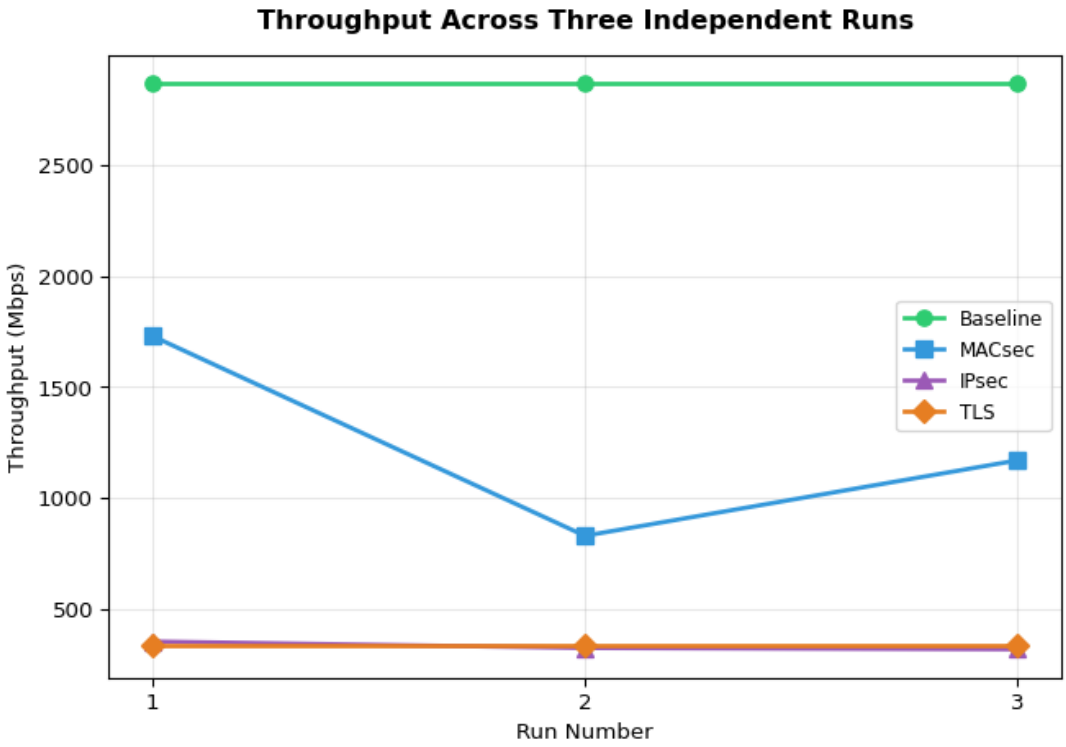

Figure 9: Throughput across three independent measurement runs per protocol, illustrating measurement variance.

data centre applications such as financial trading systems and real-time telemetry, MACsec's lower jitter represents a meaningful operational advantage.

4.2.4. CPU Utilisation Analysis

Figure 14 presents CPU utilisation during throughput testing for each protocol.

MACsec exhibited the highest CPU utilisation at 96.10%, substantially above the baseline of 73.90% and IPsec at 72.12%. This result reflects the high throughput achieved by MACsec (1243 Mbps) — the CPU was heavily loaded processing a much larger volume of frames per second than IPsec or TLS. When normalised against throughput, MACsec's CPU efficiency per Mbps is comparable to or better than IPsec and TLS. This finding directly motivates hardware offload deployment: enterprise NICs with dedicated MACsec engines perform AES-GCM encryption entirely on the NIC chip, eliminating CPU involvement regardless of traffic volume [12].

4.2.5. Packet Overhead Analysis

Figure 15 presents the theoretical per-frame overhead for each protocol. Figure 16 shows the average frame sizes measured via `tshark`.

MACsec's fixed overhead of 32 bytes per frame is lower than IPsec's approximately 50 bytes, though higher than TLS's 21-byte minimum record overhead. At data centre scale — where links carry millions of frames per second — MACsec's 32-byte overhead represents a meaningful bandwidth efficiency advantage over IPsec.

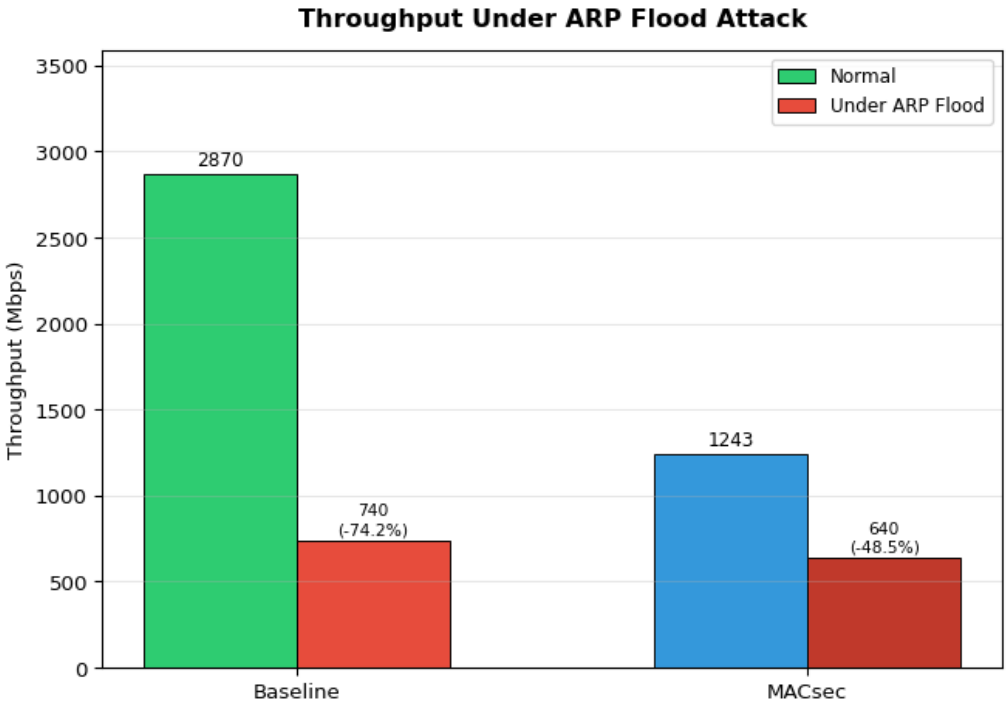

Figure 10: Throughput degradation under ARP flood attack. Unencrypted traffic degraded 74.2% while MACsec traffic degraded only 48.5% relative to their respective normal throughput values.

Table 6: Per-Frame Protocol Overhead Breakdown

Protocol	Header (bytes)	Auth Tag (bytes)	Total (bytes)
Baseline	0	0	0
MACsec	8–16	16	32
IPsec ESP	28	12	∼50
TLS 1.3	5	16	∼21

4.3. Discussion

The results collectively address the central research question: whether MACsec provides more efficient security than IPsec and TLS for zero-trust internal network protection.

Security effectiveness: MACsec is the only protocol among those evaluated capable of preventing Layer 2 attacks. The `InPktsNoTag` counter incrementing to 73 during ARP spoofing provides direct experimental evidence of attack prevention at the Data Link Layer. IPsec and TLS, operating above Layer 2, cannot detect or prevent ARP cache poisoning regardless of configuration [1, 2].

Throughput efficiency: Software MACsec achieved 1243 Mbps — significantly higher than both IPsec (333 Mbps) and TLS (339 Mbps) — despite all three relying on software-based AES-GCM-128 encryption. This advantage is attributable to MACsec's lower stack position and simpler per-frame processing pipeline. Hardware-accelerated MACsec deployments would further improve throughput to near line rate with negligible CPU overhead [12].

Attack resilience: The most operationally significant finding is MACsec's throughput

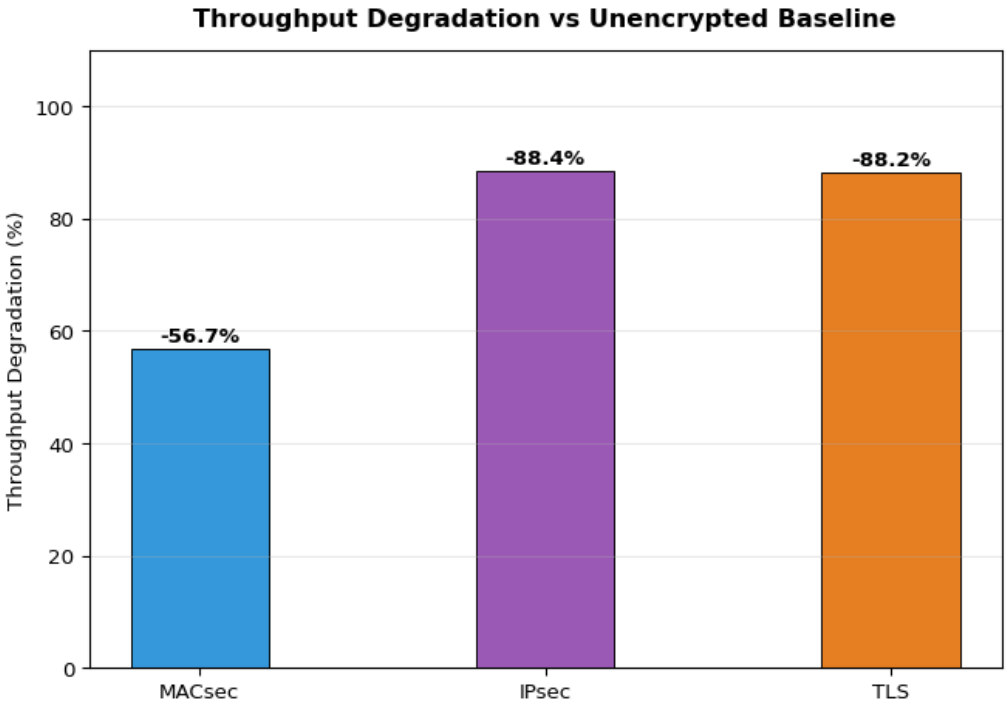

Figure 11: Throughput degradation percentage relative to unencrypted baseline for each encryption protocol.

resilience under ARP flood attack. Unencrypted traffic degraded 74.2% under attack while MACsec traffic degraded 48.5%. This demonstrates that MACsec's frame-level authentication reduces the per-frame processing burden of invalid traffic, providing a meaningful operational advantage in adversarial network conditions.

Latency and jitter: MACsec exhibited lower latency and jitter than both IPsec and TLS, consistent with its Layer 2 operation and simpler processing path. For latency-sensitive data centre workloads, this represents a meaningful advantage.

CPU utilisation: MACsec's high CPU utilisation (96.10%) reflects its high throughput rather than inefficiency — it is processing approximately 3.7 times more data per second than IPsec while consuming similar CPU resources. This limitation is architectural to software implementations and is eliminated entirely by hardware offload [12].

Conclusion of analysis: MACsec provides superior security coverage, higher throughput, lower latency, and lower jitter than both IPsec and TLS in the evaluated software deployment context. Its fixed 32-byte per-frame overhead is lower than IPsec and its exclusive ability to prevent Layer 2 attacks makes it the most appropriate encryption mechanism for zero-trust data centre environments. The primary limitation of software MACsec — CPU utilisation under high traffic — is resolved in production through hardware NIC offload engines, which are standard in enterprise-grade network interface cards [12].

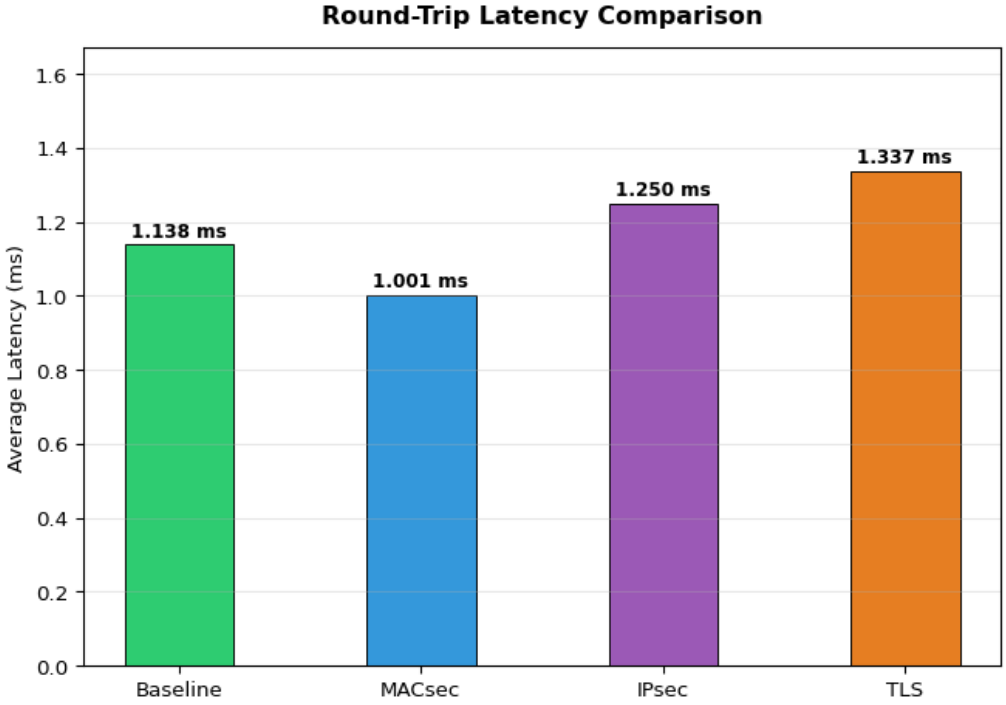

Figure 12: Average round-trip latency measured using 100 ICMP echo requests per configuration.

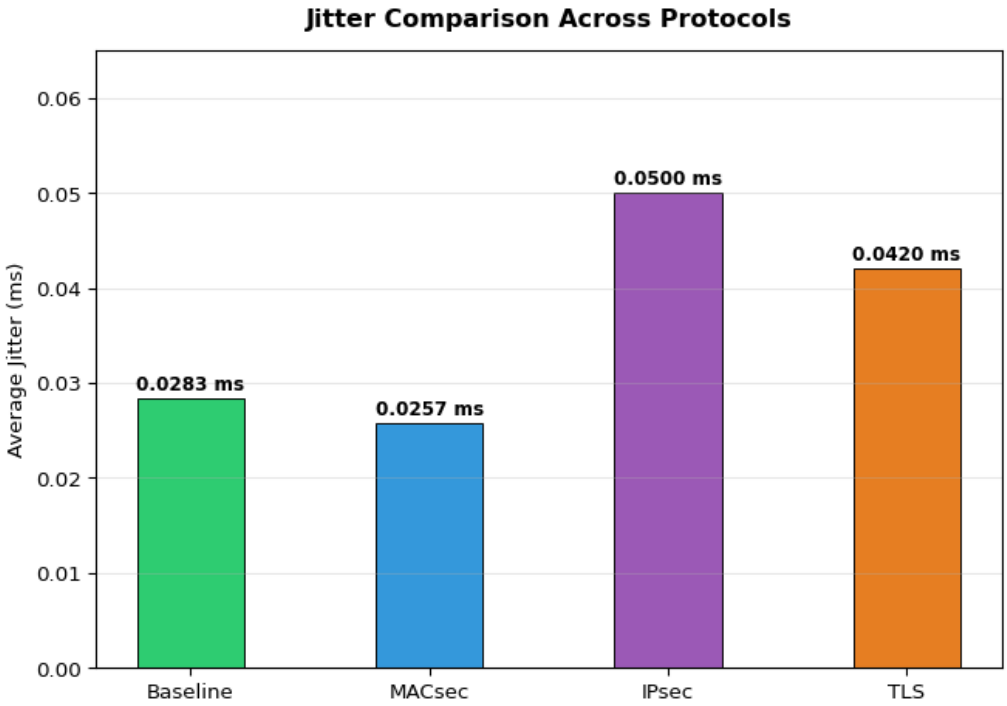

Figure 13: Average jitter across three UDP iperf3 runs per configuration.

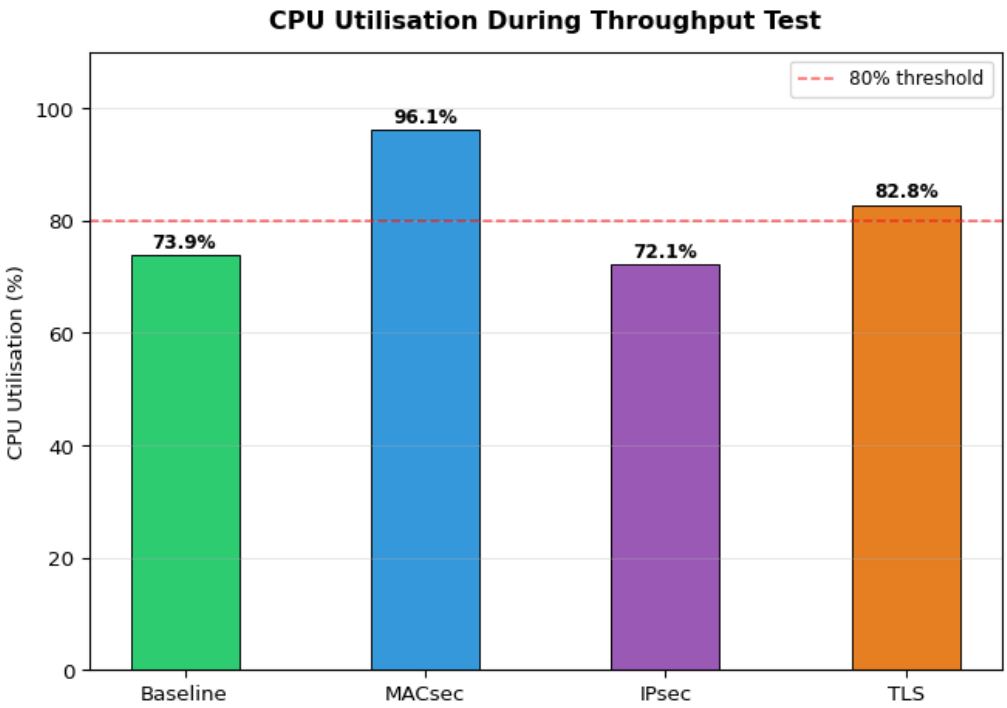

Figure 14: CPU utilisation measured via `mpstat` during 30-second iperf3 throughput tests.

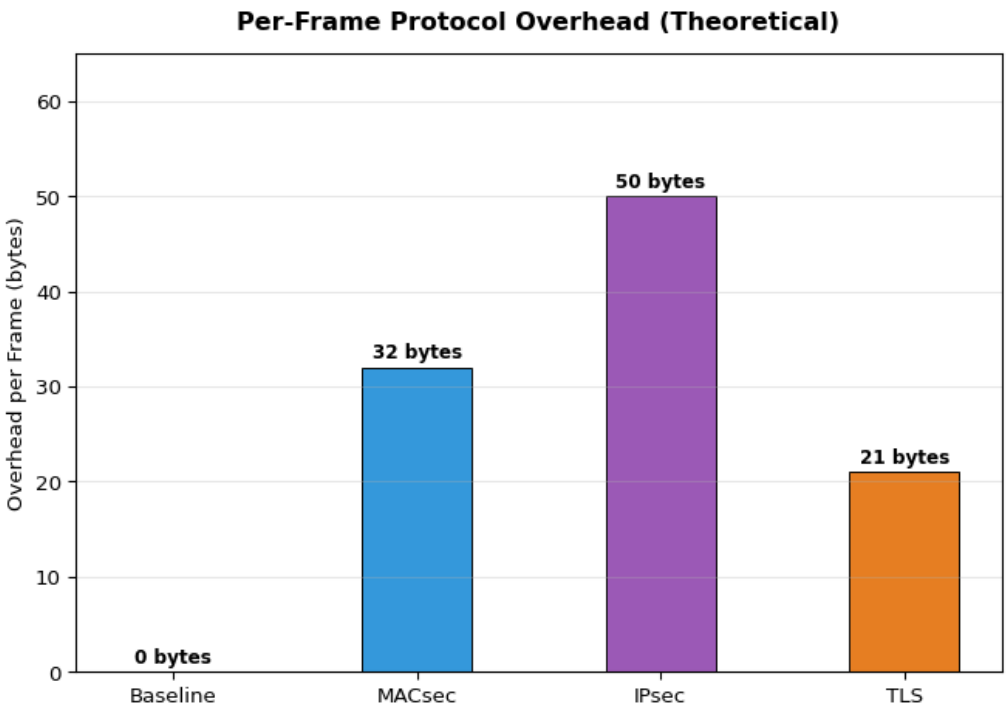

Figure 15: Theoretical per-frame protocol overhead based on header specifications [3, 1, 2].

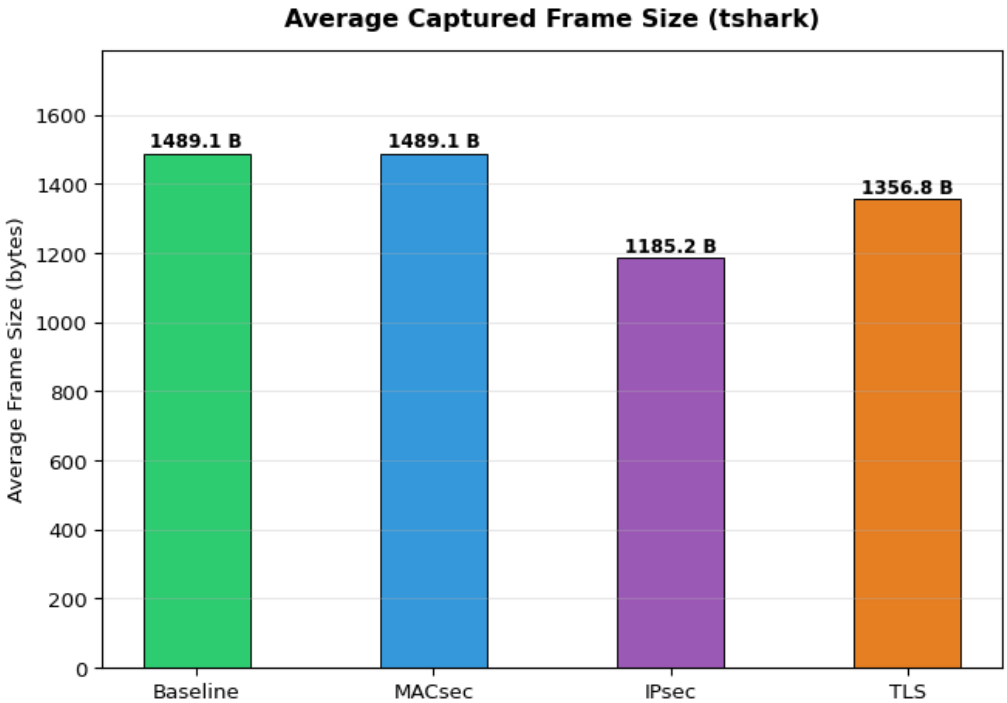

Figure 16: Average frame size captured via `tshark` during iperf3 throughput tests for each protocol.

Conclusions and Future Work

5.1. Summary of Findings

This work addressed the absence of independent, controlled experimental data comparing MACsec's security effectiveness and performance overhead against IPsec and TLS on commodity hardware. Three objectives were pursued: demonstrating Layer 2 attack exploitability and MACsec's prevention capability; benchmarking all three protocols across five performance metrics; and characterising software MACsec's CPU behaviour under attack-flood conditions.

The attack demonstrations confirm that ARP spoofing and replay attacks are trivially executable on unprotected networks and that MACsec completely prevents both, a capability that IPsec and TLS do not and cannot provide due to their operation above Layer 2. The performance benchmarks establish that software MACsec achieves significantly higher throughput than software IPsec while introducing lower latency and jitter, owing to its shorter processing path within the kernel. MACsec's per-frame overhead of 32 bytes is less than IPsec transport mode's approximate 50 bytes, further supporting its efficiency in hop-by-hop encryption scenarios [3, 1].

The CPU stress tests identify software MACsec's primary production limitation: susceptibility to CPU exhaustion under ARP flood attack. This underscores the recommendation that MACsec deployment in high-throughput or adversarial environments should use hardware-accelerated NICs and switches with dedicated AES-GCM engines [12, 11], where the NIC absorbs all cryptographic load independently of the host CPU.

5.2. Future Work

Several directions extend this work. First, repeating experiments on hardware-accelerated MACsec NICs (Intel X710, Mellanox ConnectX-5) would quantify the performance improvement achievable through offload and establish an upper bound on MACsec throughput. Second, integration of MACsec with SDN controllers such as OpenDaylight or ONOS to automate key distribution at scale would address the operational complexity identified in Section 3. Third, evaluation of MACsec under combined Layer 2 and Layer 3 attack scenarios, including MAC flooding and IP fragmentation attacks, would provide a more comprehensive security profile. Finally, deployment of MACsec in a larger-scale testbed with multiple switches and VLANs would characterise the scalability limits of the MKA key management protocol under realistic data centre topologies.

References

[1] S. Kent and K. Seo, "Security Architecture for the Internet Protocol," IETF RFC 4301, Dec. 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4301

[2] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," IETF RFC 8446, Aug. 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8446

[3] IEEE, "IEEE Standard for Local and Metropolitan Area Networks, Media Access Control (MAC) Security," *IEEE Std 802.1AE-2018*, 2018.

[4] IEEE, "IEEE Standard for Local and Metropolitan Area Networks, Port-Based Network Access Control," *IEEE Std 802.1X-2010*, 2010.

[5] J. Kindervag, "Build Security Into Your Network's DNA: The Zero Trust Network Architecture," Forrester Research, Nov. 2010.

[6] R. Ward and B. Beyer, "BeyondCorp: A New Approach to Enterprise Security," *USENIX ;login:*, vol. 39, no. 6, pp. 6–11, Dec. 2014.

[7] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, Aug. 2020.

[8] D. Song, "dsniff: A collection of tools for network auditing and penetration testing," 2001. [Online]. Available: https://www.monkey.org/~dugsong/dsniff/

[9] D. Song, "arpspoof," in *dsniff Network Auditing Toolkit*, 2001.

[10] D. Song, "arpspoof utility," in *dsniff Network Auditing Toolkit*, 2001.

[11] Cisco Systems, "MACsec Encryption Configuration Guide," Cisco IOS XE Release 3SE, 2022. [Online]. Available: https://www.cisco.com

[12] Intel Corporation, "Intel Ethernet Controllers and Adapters: MACsec Offload," Intel Technical Brief, 2021.

[13] J. Rouzaud-Cornabas, "MACsec: a different solution to encrypt network traffic," Linux Foundation Networking, 2016.

[14] J. Rouzaud-Cornabas, "Linux kernel MACsec implementation," Linux Foundation Networking, 2016.

[15] IEEE, "IEEE Std 802.1AE-2018: MAC Security," 2018.

[16] OpenSSL Project, "OpenSSL 3.x Documentation," 2023. [Online]. Available: https://www.openssl.org/docs/

[17] T. Høiland-Jørgensen *et al.*, "The eXpress data path: Fast programmable packet processing in the operating system kernel," in *Proc. ACM CoNEXT*, 2018, pp. 54–66.

[18] W. Stallings, *Data and Computer Communications*, 10th ed. Pearson, 2013.

[19] J. Kurose and K. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.